AI Assisted Coding

Assignment 16.3

Name: ch. koushik

Hall ticket no: 2303A51938

Batch no: 19

Task 1:

Prompt:

Design a Hospital Management database in SQLite by creating three related tables: Doctors, Patients, and Appointments. The tables should include primary keys, foreign keys, NOT NULL constraints, and unique constraints where necessary. Write SQL queries using joins to display all appointments for a specific doctor, retrieve the full history of a patient, and count the number of distinct patients treated by each doctor. Insert sample data and include SELECT queries to verify the database.

Code & Output:

```sql
# Task- 1 :
DROP TABLE IF EXISTS Appointments;
DROP TABLE IF EXISTS Doctors;
DROP TABLE IF EXISTS Patients;
CREATE TABLE Doctors (
    doctor_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    specialty TEXT NOT NULL
);
CREATE TABLE Patients (
    patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    date_of_birth TEXT NOT NULL
);
CREATE TABLE Appointments (
    appointment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    doctor_id INTEGER,
    patient_id INTEGER,
    appointment_date TEXT,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);
INSERT INTO Doctors (first_name, last_name, specialty)
VALUES ('Rahul', 'Sharma', 'Cardiology'),
       ('Anita', 'Reddy', 'Neurology');

INSERT INTO Patients (first_name, last_name, date_of_birth)
VALUES ('Kiran', 'Kumar', '1995-06-10'),
       ('Priya', 'Rao', '2000-03-21');
INSERT INTO Appointments (doctor_id, patient_id, appointment_date)
VALUES (1,1,'2024-07-01'),
       (1,2,'2024-07-02'),
       (2,1,'2024-07-03');
SELECT * FROM Doctors;
SELECT * FROM Patients;
SELECT * FROM Appointments;
SELECT a.appointment_id, d.first_name, p.first_name, a.appointment_date
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
JOIN Patients p ON a.patient_id = p.patient_id
WHERE a.doctor_id = 1;
SELECT p.first_name, p.last_name, a.appointment_date
FROM Appointments a
JOIN Patients p ON a.patient_id = p.patient_id
WHERE p.patient_id = 1;
SELECT d.first_name, COUNT(DISTINCT a.patient_id)
FROM Doctors d
LEFT JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.doctor_id;
```

Output 1 – Doctors Table

doctor_id	first_name	last_name	specialty
1	Rahul	Sharma	Cardiology
2	Anita	Reddy	Neurology

Output 2 – Patients Table

patient_id	first_name	last_name	date_of_birth
1	Kiran	Kumar	1995-06-10
2	Priya	Rao	2000-03-21

Output 3 – Appointments Table

appointment_id	doctor_id	patient_id	appointment_date
1	1	1	2024-07-01
2	1	2	2024-07-02
3	2	1	2024-07-03

Output 4 – Appointments for Doctor

appointment_id	doctor_name	patient_name	date
1	Rahul	Kiran	2024-07-01
2	Rahul	Priya	2024-07-02

Explanation:

This database is designed using three related tables connected through foreign keys to maintain relationships between doctors, patients, and appointments. SQL joins are used to retrieve appointment details and patient history efficiently. Aggregate functions are used to count the number of patients treated by each doctor. The database structure follows normalization to avoid redundancy and maintain data integrity.

Task 2:

Prompt:

Create an E-commerce database system using SQLite with tables Users, Products, Orders, and OrderDetails. Include primary keys, foreign keys, and necessary constraints. Write SQL queries to retrieve orders placed by a specific user, identify the most purchased product, and calculate total revenue for a particular month. Insert sample data and verify using SELECT queries.

Code & Output:

```sql
# Task- 2:
DROP TABLE IF EXISTS OrderDetails;
DROP TABLE IF EXISTS Orders;
DROP TABLE IF EXISTS Products;
DROP TABLE IF EXISTS Users;
CREATE TABLE Users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT NOT NULL,
    email TEXT UNIQUE
);
CREATE TABLE Products (
    product_id INTEGER PRIMARY KEY AUTOINCREMENT,
    product_name TEXT NOT NULL,
    price REAL NOT NULL,
    stock INTEGER NOT NULL
);
CREATE TABLE Orders (
    order_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER,
    order_date TEXT,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
CREATE TABLE OrderDetails (
    order_detail_id INTEGER PRIMARY KEY AUTOINCREMENT,
    order_id INTEGER,
    product_id INTEGER,
    quantity INTEGER,
    price REAL,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
INSERT INTO Users (username, email) VALUES ('charan', 'charan@gmail.com');
INSERT INTO Products (product_name, price, stock)
VALUES ('Laptop', 50000, 5),
       ('Mouse', 500, 20);
INSERT INTO Orders (user_id, order_date) VALUES (1,'2024-01-10');
INSERT INTO OrderDetails (order_id, product_id, quantity, price)
VALUES (1,1,1,50000),
       (1,2,2,500);
SELECT * FROM Users;
SELECT * FROM Products;
SELECT * FROM Orders;
SELECT * FROM OrderDetails;
SELECT o.order_id, od.product_id, od.quantity
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
WHERE o.user_id = 1;
SELECT p.product_name, SUM(od.quantity)
FROM Products p
JOIN OrderDetails od ON p.product_id = od.product_id
GROUP BY p.product_id
ORDER BY SUM(od.quantity) DESC
LIMIT 1;
SELECT SUM(od.quantity * od.price)
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
WHERE strftime('%Y-%m', o.order_date) = '2024-01';
```

Users Table

user_id	username	email
1	charan	charan@gmail.com

Products Table

product_id	product_name	price	stock
1	Laptop	50000	5
2	Mouse	500	20

Orders Table

order_id	user_id	order_date
1	1	2024-01-10

OrderDetails Table

order_detail_id	order_id	product_id	quantity	price
1	1	1	1	50000
2	1	2	2	500

Most Purchased Product Output

product_name	total_quantity
Mouse	2

Explanation:

The E-commerce database consists of multiple related tables where OrderDetails connects Orders and Products, maintaining proper normalization. SQL joins are used to retrieve order information and analyze product sales. Aggregate functions help identify the most purchased product and calculate total revenue. This structure avoids data duplication and improves query efficiency.

Task 3:

Prompt:

Design a Library Management database using SQLite with three tables: Books, Members, and Loans. Include primary keys, foreign keys, and constraints such as NOT NULL and UNIQUE. Write SQL queries to retrieve books that are currently issued, find overdue books where the loan period is more than 30 days, and count the number of books borrowed by each member. Insert sample data and ensure the database is normalized.

Code & Output:

```sql
# Task- 3:
DROP TABLE IF EXISTS Loans;
DROP TABLE IF EXISTS Books;
DROP TABLE IF EXISTS Members;
CREATE TABLE Books (
    BookID INTEGER PRIMARY KEY AUTOINCREMENT,
    Title TEXT NOT NULL,
    Author TEXT NOT NULL,
    PublishedYear INTEGER
);
CREATE TABLE Members (
    MemberID INTEGER PRIMARY KEY AUTOINCREMENT,
    FirstName TEXT NOT NULL,
    LastName TEXT NOT NULL,
    Email TEXT UNIQUE
);
CREATE TABLE Loans (
    LoanID INTEGER PRIMARY KEY AUTOINCREMENT,
    BookID INTEGER,
    MemberID INTEGER,
    LoanDate DATE,
    ReturnDate DATE,
    FOREIGN KEY (BookID) REFERENCES Books(BookID),
    FOREIGN KEY (MemberID) REFERENCES Members(MemberID)
);
INSERT INTO Books (Title, Author, PublishedYear)
VALUES ('Database Systems', 'Navathe', 2018),
       ('Operating Systems', 'Galvin', 2016);
INSERT INTO Members (FirstName, LastName, Email)
VALUES ('Ravi', 'Kumar', 'ravi@gmail.com'),
       ('Sneha', 'Reddy', 'sneha@gmail.com');
INSERT INTO Loans (BookID, MemberID, LoanDate)
VALUES (1,1,'2024-05-01'),
       (2,2,'2024-05-15');
SELECT b.Title, m.FirstName, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
JOIN Members m ON l.MemberID = m.MemberID
WHERE l.ReturnDate IS NULL;
SELECT b.Title, l.LoanDate
FROM Loans l
JOIN Books b ON l.BookID = b.BookID
WHERE julianday('now') - julianday(l.LoanDate) > 30;
SELECT m.FirstName, COUNT(l.LoanID)
FROM Members m
LEFT JOIN Loans l ON m.MemberID = l.MemberID
GROUP BY m.MemberID;
```

Books Issued

Title	Member	LoanDate
Database Systems	Ravi	2024-05-01
Operating Systems	Sneha	2024-05-15

Overdue Books

Title	LoanDate
Database Systems	2024-05-01

Books Loaned Per Member

Member	BooksLoaned
Ravi	1
Sneha	1

Explanation:

The library database is designed using three related tables connected through foreign keys. SQL joins are used to retrieve issued books, overdue books, and borrowing details. Aggregate functions help calculate the number of books borrowed by each member. The database structure follows normalization principles to reduce redundancy and improve data integrity.

Task 4:

Prompt:

Analyze and optimize SQL queries for better performance. First write a query using a subquery to retrieve books written by UK authors, then rewrite the same query using a JOIN for optimization. Ensure both queries return the same result and suggest indexing to improve performance.

Code & Output:

```sql
# Task- 4:
DROP TABLE IF EXISTS Books;
DROP TABLE IF EXISTS Authors;
CREATE TABLE Authors (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT,
    country TEXT
);
CREATE TABLE Books (
    book_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT,
    author_id INTEGER,
    FOREIGN KEY (author_id) REFERENCES Authors(id)
);
INSERT INTO Authors (name, country)
VALUES ('Author A', 'UK'),
       ('Author B', 'USA'),
       ('Author C', 'UK');
INSERT INTO Books (title, author_id)
VALUES ('Book One', 1),
       ('Book Two', 2),
       ('Book Three', 3);
-- Original Query (Subquery)
SELECT title
FROM Books
WHERE author_id IN (
    SELECT id FROM Authors WHERE country = 'UK'
);
-- Optimized Query (JOIN)
SELECT b.title
FROM Books b
JOIN Authors a ON b.author_id = a.id
WHERE a.country = 'UK';
-- Indexing
CREATE INDEX idx_books_author_id ON Books(author_id);
CREATE INDEX idx_authors_country ON Authors(country);
```

title
Book One
Book Three

Both original query and optimized query should return same result.

Explanation:

The original query uses a subquery which may scan the database multiple times and reduce performance. The optimized query uses a JOIN operation which improves execution speed and scalability. Indexing important columns such as author_id and country further improves query performance. Both queries return the same results but the optimized query executes faster.

Task 5:

Prompt:

Design a University Course Registration database with four tables: Students, Faculty, Courses, and Registrations. Include primary keys, foreign keys, and constraints. Write SQL queries to list students enrolled in a course, find faculty teaching more than two courses, and retrieve the course with the highest number of registrations.

Code & Output:

Explanation:

```sql
# Task -5 :
DROP TABLE IF EXISTS Registrations;
DROP TABLE IF EXISTS Courses;
DROP TABLE IF EXISTS Faculty;
DROP TABLE IF EXISTS Students;
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT,
    last_name TEXT,
    email TEXT UNIQUE
);
CREATE TABLE Faculty (
    faculty_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT,
    last_name TEXT,
    email TEXT UNIQUE
);
CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_name TEXT,
    faculty_id INTEGER,
    FOREIGN KEY (faculty_id) REFERENCES Faculty(faculty_id)
);
CREATE TABLE Registrations (
    registration_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER,
    course_id INTEGER,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
INSERT INTO Faculty (first_name, last_name, email)
VALUES ('Dr', 'Rao', 'rao@university.com');
INSERT INTO Students (first_name, last_name, email)
VALUES ('Charan', 'Yadav', 'charan@gmail.com');
INSERT INTO Courses (course_name, faculty_id)
VALUES ('Database Systems', 1);
INSERT INTO Registrations (student_id, course_id)
VALUES (1,1);
SELECT s.first_name, s.last_name
FROM Students s
JOIN Registrations r ON s.student_id = r.student_id
WHERE r.course_id = 1;
SELECT f.first_name, COUNT(c.course_id)
FROM Faculty f
JOIN Courses c ON f.faculty_id = c.faculty_id
GROUP BY f.faculty_id
HAVING COUNT(c.course_id) > 2;
SELECT c.course_name, COUNT(r.registration_id)
FROM Courses c
JOIN Registrations r ON c.course_id = r.course_id
GROUP BY c.course_id
ORDER BY COUNT(r.registration_id) DESC
LIMIT 1;
```

Students in Course

first_name	last_name
Charan	Yadav

Course with Highest Registration

course_name	registration_count
Database Systems	1

Explanation:

The university course registration database uses multiple related tables where the Registrations table connects Students and Courses to manage many-to-many relationships. SQL joins are used to retrieve enrollment details and faculty course information. Aggregate functions help identify popular courses and faculty workload. The database structure is normalized to maintain consistency and reduce data redundancy.